LAB-4

WORD SCRAMBLE

VibeCoding Lab Report

NAME: REHAN S. BAWAKHAN

SRN: PES2UG25CS820

SECTION: 5J

GitHub Repository

https://github.com/rehanbawakhan/SE-Lab/tree/main/Lab-4

Base project: https://github.com/SETAPESU26/62_scramble

1. Project Overview

Word Scramble Arena is a Python anagram puzzle game built with Pygame. The original game picks a random word from a built-in list, shuffles its letters (guaranteeing the shuffled version differs from the original), and asks the player to type the unscrambled word into a custom TextBox widget. Guesses are submitted with Enter or the SUBMIT button, and the game shows a score and colour-coded feedback messages.

The provided code contained one deliberate bug: the guess was validated against the scrambled display text instead of the secret word. The Lab-4 update fixes that bug and adds a progressive hint system, a round countdown timer, and interactive letter tiles, while keeping the original structure (main.py → GameEngine → TextBox).

2. Bug Fix and Implemented Features

Task 1 – Guess Comparison Bug (Fixed)

In submit_guess(), the check was **is_correct = (guess == self.scrambled_word)**. Typing the real word was rejected as wrong, while re-typing the scrambled letters was accepted. It now reads **is_correct = (guess == self.secret_word)**. Input is still stripped and upper-cased, empty submissions still show a prompt without penalty, and a wrong guess still clears the box without advancing the round.

Task 2 – Progressive Hint System

- Press the **HINT** button or the **Tab** key to reveal one letter of the secret word in its correct position.
- The matching tile is swapped into that slot and **locked** (shown in green), so it cannot be moved again.
- Proportional score penalty: a word is worth 10 points, and each hint removes one letter's share – **worth = 10 × (letters − hints) / letters** (minimum 1). The HUD shows hints used and current worth.
- Hints stop when only one letter is left unrevealed, so the player always solves the final step.

Task 3 – Round Countdown Timer

- Each round lasts **30 seconds**. A timer bar under the score drains in real time, with the seconds remaining shown beside it.
- The bar changes colour: green (> 50%), amber (20–50%), red (< 20%).
- Time is tracked with pygame.time.get_ticks() in update(), so it is frame-rate independent and does not block the game loop.
- When time runs out the game shows **"TIME'S UP! The word was …"** and automatically advances to a new word.

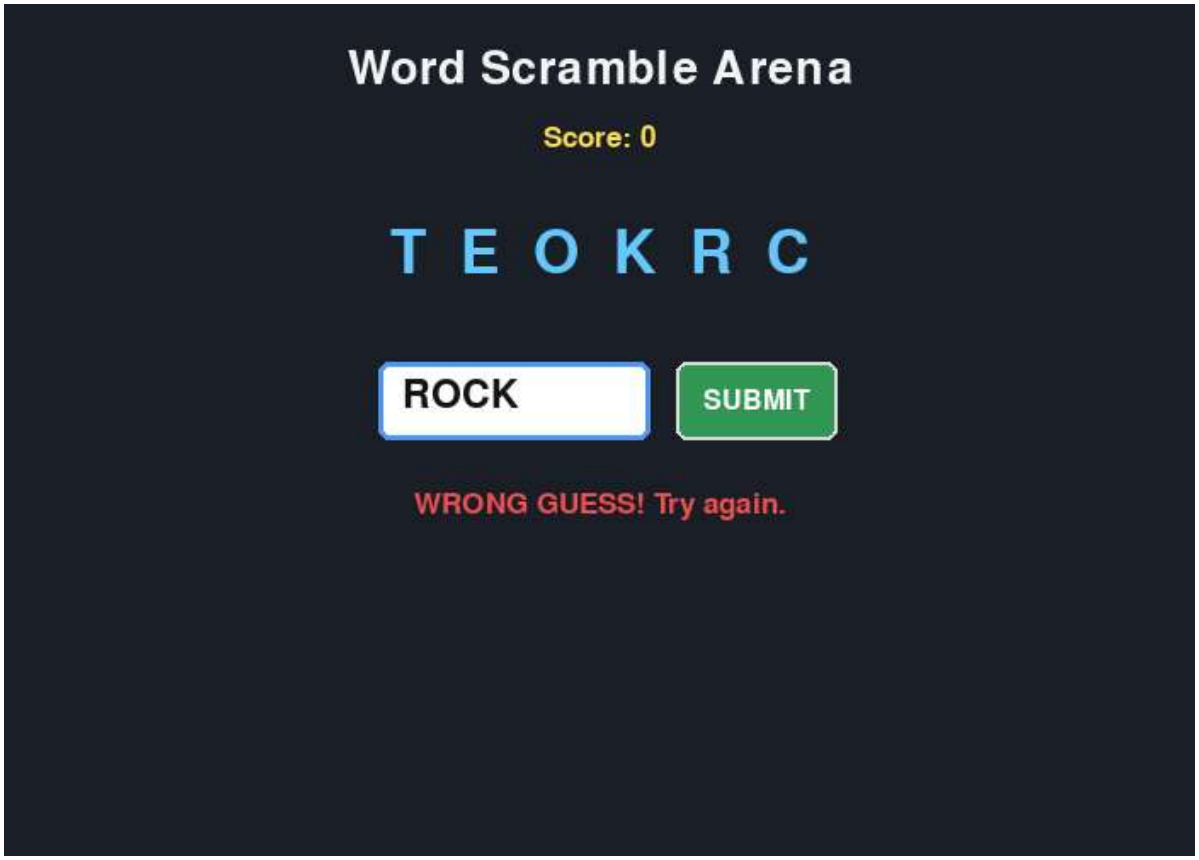
Task 4 – Interactive Letter Tiles

- The static scrambled text is replaced by graphical letter tiles.
- Click one tile to select it (it lifts and turns yellow), then click another to **swap** them. Clicking the same tile again deselects it.
- **SHUFFLE TILES** reshuffles all unlocked tiles; **USE TILE ORDER** copies the current tile arrangement into the input box so it can be submitted.
- Hint-locked tiles stay fixed during swaps and shuffles.

Other Fixes and Controls

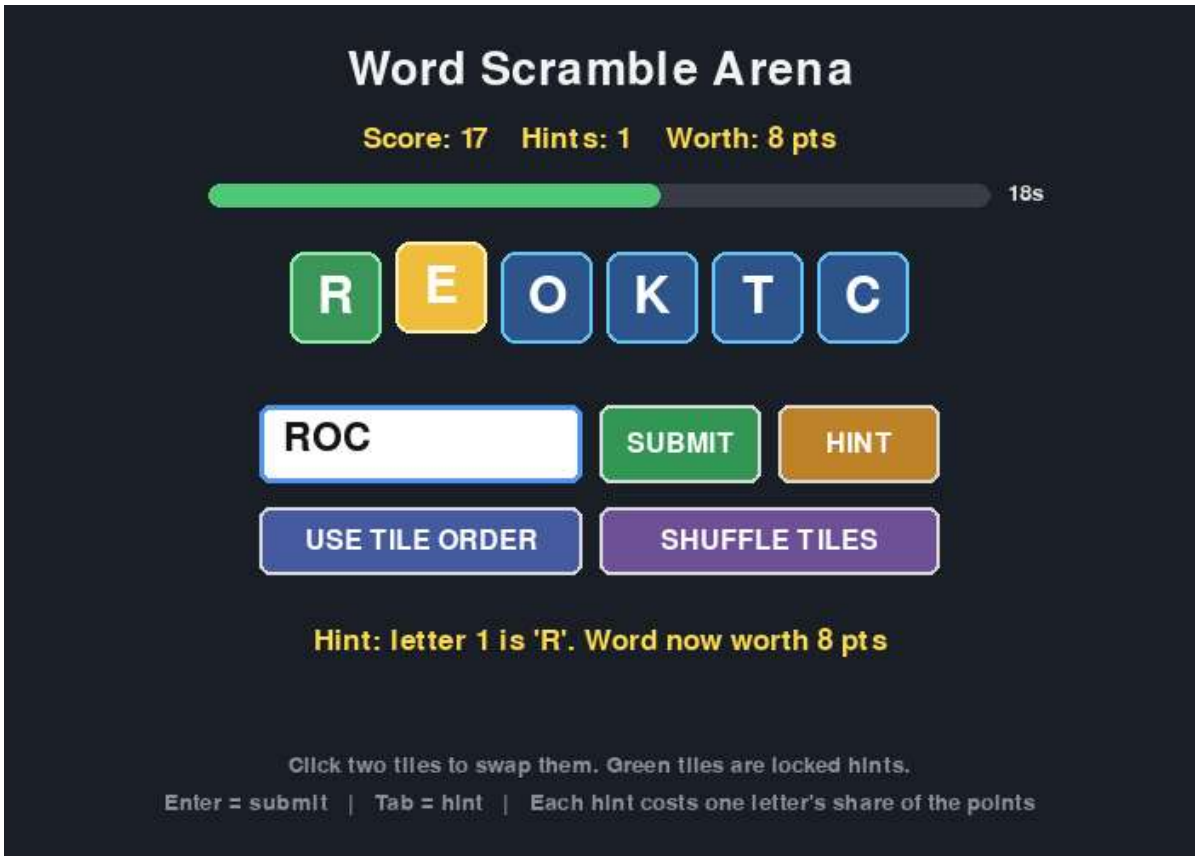
- Window caption corrected from "Anvil Dodge - Pygame Edition" to "Word Scramble Arena".
- Clicking a game button or tile keeps keyboard focus on the text box, so typing still works.
- Controls: type letters to guess; **Enter** / SUBMIT to submit; **Tab** / HINT for a hint; mouse click on tiles to swap; Backspace to delete.
- text_box.py is unchanged.

3. Gameplay Evidence – Before Modification



The correct word (ROCKET) is typed and submitted, but the original code rejects it with "WRONG GUESS! Try again." because it compares against the scrambled text. Letters are shown as a static string.

4. Gameplay Evidence – After Modification



Updated game: the timer bar is counting down (18s left), one hint has been used (green locked tile "R", word now worth 8 pts), a tile is selected for swapping (yellow), and the new HINT, USE TILE ORDER and SHUFFLE TILES controls are visible.

5. Chat History / LLM Assistance

Claude share link: https://claude.ai/share/286ef192-0d95-498d-a9a4-72e58bed4534

LLM used: Claude (Anthropic). The full transcript is also attached as chat_history.pdf.

5.1 Understanding the Existing Project

Prompt summary: Shared the lab handout and the repository link (SETAPESU26/62_scramble). The assistant cloned the repo and reviewed README.md, main.py, game/game_engine.py and game/text_box.py. The existing game already had random word selection, a scrambling function that guarantees a different order, the TextBox input widget, Enter/SUBMIT submission, scoring and colour-coded feedback.

5.2 Locating the Bug (Task 1)

The README described correct words being rejected and scrambled strings being accepted. Reading submit_guess() showed the comparison used self.scrambled_word. The fix was a one-line change to compare with self.secret_word, keeping the existing empty-input and wrong-guess behaviour.

5.3 Hint System (Task 2)

Design discussed: reveal letters in their correct positions, lock revealed tiles, and apply a proportional penalty rather than a flat one. Chosen formula: 10 × (n − hints) / n with a minimum of 1 point, with hints stopped at one remaining letter. HINT button and Tab shortcut added.

5.4 Countdown Timer (Task 3)

A 30-second round timer was added using pygame.time.get_ticks() inside update() so it is independent of frame rate. A colour-changing bar was drawn in render(); on timeout the word is revealed and next_round() resets the timer, hints and tiles.

5.5 Interactive Letter Tiles (Task 4)

The static label was replaced by clickable tiles laid out from the word length. Click-to-select / click-to-swap was chosen over dragging for reliability. SHUFFLE TILES and USE TILE ORDER were added, and locked hint tiles are excluded from swaps and shuffles.

5.6 Testing

The updated engine was tested headlessly (SDL dummy driver) before delivery. Results:

```
scrambled word rejected : True
correct word accepted   : True
hint locks correct letter: True  (word worth 7 after 2 hints)
max hints for 6 letters : 5
tile swap works         : True
USE TILE ORDER fills box: True
timeout reveals + resets: True  ("TIME'S UP! The word was 'PUZZLE'.")
```

The game was then run locally on Windows to record the before/after gameplay clips.

5.7 Environment Debugging

- **pip install pygame failed** on Python 3.14 (no prebuilt wheel; source build error "No module named setuptools._distutils.msvccompiler"). Fixed by installing **pygame-ce**, the maintained fork, which supports 3.14 and is still imported as pygame – no code changes required.

- **"Python was not found"** – the python command pointed to the Microsoft Store alias. Fixed by running the game with the **py** launcher (py main.py).

5.8 Submission

The updated code was copied into a Lab-4 folder in the personal lab repository together with the before/after gameplay videos and the chat history, then committed and pushed. No pull request was opened to the course organisation repository.

6. Updated Code

game/game_engine.py

```
import random
import pygame
from game.text_box import TextBox

ROUND_TIME = 30.0        # seconds per round
WORD_POINTS = 10         # points for a word solved with no hints
TILE_SIZE = 54
TILE_GAP = 8


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.words = ["PYTHON", "PYGAME", "PLANET", "ROCKET", "GALAXY", "STREAM", "PUZZLE", "ALGORITHM"]
        self.secret_word = ""
        self.scrambled_word = ""

        self.score = 0
        self.feedback_msg = "Unscramble the letters above!"
        self.feedback_color = (210, 215, 225)

        # Hint state
        self.hints_used = 0
        self.locked = set()        # tile indices fixed in place by hints

        # Tile state
        self.tiles = []            # current letter order shown as tiles
        self.tile_rects = []
        self.selected_tile = None

        # Timer state
        self.time_left = ROUND_TIME
        self.last_tick = pygame.time.get_ticks()

        # Layout
        self.input_box = TextBox(width // 2 - 200, 235, 190, 46)
        self.submit_btn = pygame.Rect(width // 2 + 0, 235, 95, 46)
        self.hint_btn = pygame.Rect(width // 2 + 105, 235, 95, 46)
        self.use_tiles_btn = pygame.Rect(width // 2 - 200, 295, 190, 40)
        self.shuffle_btn = pygame.Rect(width // 2 + 0, 295, 200, 40)

        self.font_title = pygame.font.SysFont(None, 40)
        self.font_tile = pygame.font.SysFont(None, 44)
        self.font_msg = pygame.font.SysFont(None, 26)
        self.font_btn = pygame.font.SysFont(None, 24)
        self.font_small = pygame.font.SysFont(None, 20)

        self.next_round()

    # ---------------------------------------------------------------- setup
    def scramble_string(self, word):
```

```
    letters = list(word)
    while True:
        random.shuffle(letters)
        shuffled = "".join(letters)
        if shuffled != word or len(word) <= 1:
            return shuffled

def next_round(self):
    self.secret_word = random.choice(self.words)
    self.scrambled_word = self.scramble_string(self.secret_word)
    self.tiles = list(self.scrambled_word)
    self.selected_tile = None
    self.locked = set()
    self.hints_used = 0
    self.time_left = ROUND_TIME
    self.last_tick = pygame.time.get_ticks()
    self.input_box.clear()
    self._layout_tiles()

def _layout_tiles(self):
    n = len(self.tiles)
    total = n * TILE_SIZE + (n - 1) * TILE_GAP
    start_x = self.width // 2 - total // 2
    self.tile_rects = [
        pygame.Rect(start_x + i * (TILE_SIZE + TILE_GAP), 145, TILE_SIZE, TILE_SIZE)
        for i in range(n)
    ]

# ---------------------------------------------------------------- scoring
def round_value(self):
    """Proportional penalty: each hint removes one letter's share of the points."""
    n = len(self.secret_word)
    return max(1, round(WORD_POINTS * (n - self.hints_used) / n))

def submit_guess(self):
    guess = self.input_box.text.strip().upper()
    if not guess:
        self.feedback_msg = "Type a word before submitting!"
        self.feedback_color = (240, 170, 50)
        return

    # Task 1 fix: compare against the secret word, not the scrambled display
    is_correct = (guess == self.secret_word)

    if is_correct:
        gained = self.round_value()
        self.score += gained
        self.feedback_msg = f"CORRECT! '{self.secret_word}' is right. +{gained}"
        self.feedback_color = (80, 230, 110)
        self.next_round()
    else:
        self.feedback_msg = "WRONG GUESS! Try again."
        self.feedback_color = (240, 80, 80)
        self.input_box.clear()

# ---------------------------------------------------------------- hints
def use_hint(self):
```

```
    n = len(self.secret_word)
    unrevealed = [i for i in range(n) if i not in self.locked]
    if len(unrevealed) <= 1:
        self.feedback_msg = "No more hints for this word!"
        self.feedback_color = (240, 170, 50)
        return

    pos = random.choice(unrevealed)
    needed = self.secret_word[pos]

    # Move a matching, unlocked tile into the correct position, then lock it
    if self.tiles[pos] != needed:
        for j in range(n):
            if j not in self.locked and j != pos and self.tiles[j] == needed:
                self.tiles[pos], self.tiles[j] = self.tiles[j], self.tiles[pos]
                break
    self.locked.add(pos)
    self.selected_tile = None
    self.hints_used += 1

    self.feedback_msg = (f"Hint: letter {pos + 1} is '{needed}'. "
                         f"Word now worth {self.round_value()} pts")
    self.feedback_color = (255, 220, 80)

# ------------------------------------------------------------------- tiles
def click_tile(self, idx):
    if idx in self.locked:
        return
    if self.selected_tile is None:
        self.selected_tile = idx
    elif self.selected_tile == idx:
        self.selected_tile = None
    else:
        a, b = self.selected_tile, idx
        self.tiles[a], self.tiles[b] = self.tiles[b], self.tiles[a]
        self.selected_tile = None

def shuffle_tiles(self):
    free = [i for i in range(len(self.tiles)) if i not in self.locked]
    letters = [self.tiles[i] for i in free]
    random.shuffle(letters)
    for i, ch in zip(free, letters):
        self.tiles[i] = ch
    self.selected_tile = None

def use_tiles(self):
    self.input_box.text = "".join(self.tiles)

# ------------------------------------------------------------------ events
def handle_event(self, event):
    self.input_box.handle_event(event)

    if event.type == pygame.KEYDOWN:
        if event.key in (pygame.K_RETURN, pygame.K_KP_ENTER):
            self.submit_guess()
        elif event.key == pygame.K_TAB:
            self.use_hint()
```

```
        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            clicked_ui = True
            if self.submit_btn.collidepoint(event.pos):
                self.submit_guess()
            elif self.hint_btn.collidepoint(event.pos):
                self.use_hint()
            elif self.use_tiles_btn.collidepoint(event.pos):
                self.use_tiles()
            elif self.shuffle_btn.collidepoint(event.pos):
                self.shuffle_tiles()
            else:
                clicked_ui = False
                for i, rect in enumerate(self.tile_rects):
                    if rect.collidepoint(event.pos):
                        self.click_tile(i)
                        clicked_ui = True
                        break
            # keep typing focus after clicking any game control
            if clicked_ui:
                self.input_box.active = True

    # ------------------------------------------------------------------ update
    def update(self):
        now = pygame.time.get_ticks()
        self.time_left -= (now - self.last_tick) / 1000.0
        self.last_tick = now

        if self.time_left <= 0:
            word = self.secret_word
            self.next_round()
            self.feedback_msg = f"TIME'S UP! The word was '{word}'."
            self.feedback_color = (240, 80, 80)

    # ------------------------------------------------------------------ render
    def _button(self, screen, rect, label, color):
        pygame.draw.rect(screen, color, rect, border_radius=6)
        pygame.draw.rect(screen, (220, 220, 220), rect, width=2, border_radius=6)
        txt = self.font_btn.render(label, True, (255, 255, 255))
        screen.blit(txt, (rect.centerx - txt.get_width() // 2, rect.centery - txt.get_height() // 2))

    def render(self, screen):
        screen.fill((26, 30, 38))

        title_surf = self.font_title.render("Word Scramble Arena", True, (245, 245, 245))
        screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 25))

        score_surf = self.font_msg.render(
            f"Score: {self.score}    Hints: {self.hints_used}    Worth: {self.round_value()} pts",
            True, (255, 220, 80))
        screen.blit(score_surf, (self.width // 2 - score_surf.get_width() // 2, 70))

        # Timer bar
        bar_w, bar_h = 460, 14
        bar_x, bar_y = self.width // 2 - bar_w // 2, 105
        frac = max(0.0, self.time_left / ROUND_TIME)
        if frac > 0.5:
            bar_color = (80, 200, 120)
```

```
        elif frac > 0.2:
            bar_color = (240, 190, 60)
        else:
            bar_color = (235, 75, 75)
        pygame.draw.rect(screen, (55, 60, 72), (bar_x, bar_y, bar_w, bar_h), border_radius=7)
        pygame.draw.rect(screen, bar_color, (bar_x, bar_y, int(bar_w * frac), bar_h), border_radius=7)
        t_surf = self.font_small.render(f"{max(0, int(self.time_left + 0.999))}s", True, (230, 230, 230))
        screen.blit(t_surf, (bar_x + bar_w + 10, bar_y - 1))

        # Letter tiles
        for i, rect in enumerate(self.tile_rects):
            if i in self.locked:
                fill, border = (60, 150, 90), (140, 240, 170)
            elif i == self.selected_tile:
                fill, border = (240, 190, 60), (255, 240, 180)
            else:
                fill, border = (45, 85, 140), (100, 200, 255)
            draw_rect = rect.move(0, -6) if i == self.selected_tile else rect
            pygame.draw.rect(screen, fill, draw_rect, border_radius=8)
            pygame.draw.rect(screen, border, draw_rect, width=2, border_radius=8)
            letter = self.font_tile.render(self.tiles[i], True, (255, 255, 255))
            screen.blit(letter, (draw_rect.centerx - letter.get_width() // 2,
                                 draw_rect.centery - letter.get_height() // 2))

        self.input_box.render(screen)

        self._button(screen, self.submit_btn, "SUBMIT", (50, 150, 85))
        self._button(screen, self.hint_btn, "HINT", (190, 130, 40))
        self._button(screen, self.use_tiles_btn, "USE TILE ORDER", (70, 90, 160))
        self._button(screen, self.shuffle_btn, "SHUFFLE TILES", (110, 80, 150))

        feedback_surf = self.font_msg.render(self.feedback_msg, True, self.feedback_color)
        screen.blit(feedback_surf, (self.width // 2 - feedback_surf.get_width() // 2, 365))

        help_lines = [
            "Click two tiles to swap them. Green tiles are locked hints.",
            "Enter = submit  |  Tab = hint  |  Each hint costs one letter's share of the points",
        ]
        for k, line in enumerate(help_lines):
            s = self.font_small.render(line, True, (140, 145, 160))
            screen.blit(s, (self.width // 2 - s.get_width() // 2, 440 + k * 22))
```

main.py

```
import pygame
from game.game_engine import GameEngine

WIDTH, HEIGHT = 700, 500
FPS = 60


def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Word Scramble Arena")
    clock = pygame.time.Clock()

    engine = GameEngine(WIDTH, HEIGHT)
```

```
    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            engine.handle_event(event)

        engine.update()
        engine.render(screen)

        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()


if __name__ == "__main__":
    main()
```

game/text_box.py (unchanged)

```
import pygame

class TextBox:
    def __init__(self, x, y, width, height):
        self.rect = pygame.Rect(x, y, width, height)
        self.text = ""
        self.active = True
        self.font = pygame.font.SysFont(None, 34)

    def handle_event(self, event):
        if event.type == pygame.MOUSEBUTTONDOWN:
            self.active = self.rect.collidepoint(event.pos)
        elif event.type == pygame.KEYDOWN and self.active:
            if event.key == pygame.K_BACKSPACE:
                self.text = self.text[:-1]
            elif event.unicode.isalpha() and len(self.text) < 15:
                self.text += event.unicode.upper()

    def clear(self):
        self.text = ""

    def render(self, surface):
        box_color = (80, 150, 245) if self.active else (100, 105, 115)
        pygame.draw.rect(surface, (255, 255, 255), self.rect, border_radius=6)
        pygame.draw.rect(surface, box_color, self.rect, width=3, border_radius=6)

        text_surf = self.font.render(self.text, True, (20, 20, 20))
        surface.blit(text_surf, (self.rect.x + 14, self.rect.y + 8))
```